SRE Technical Portfolio - Supplementary Material | SRE 技術実績 - 補足資料

Wei ZHOU | 周 偉

March 2026 | 2026 年 3 月

Contents

1 SRE Technical Portfolio - Supplementary Material **2**

2 SRE 技術実績 - 補足資料 **2**
2.1 Document Structure | ドキュメント構成 2

3 [ENGLISH VERSION] **2**
3.1 Case A: Engineering Foundation & Reliability Infrastructure 2
Context 2
SRE Actions & Impact 2
Key Achievement 3
3.2 Case B: Disaster Recovery via Zero-Dependency Deployment 3
Context 3
SRE Actions & Impact 3
Key Achievement 4
3.3 Case C: Legacy Modernization via Binary-Level Patching 4
Context 4
SRE Actions & Impact 4
Key Achievement 5
3.4 Technical Competency Matrix 5

4 [日本語版] **5**
4.1 ケース A：1 人目 SRE としてのエンジニアリング基盤と文化のゼロイチ構築 . . 5
背景 5
SRE 活動と実績 5
主要成果 6
4.2 ケース B：ゼロ依存デプロイによる災害復旧 (DR/Zero-dependency) 6
背景 6
SRE 活動と実績 6
主要成果 7
4.3 ケース C：バイナリレベルのレガシー近代化 (Modernization) 7
背景 7
SRE 活動と実績 7
主要成果 7
4.4 技術能力マトリックス 7

4.5 Summary | まとめ 8

1 SRE Technical Portfolio - Supplementary Material

2 SRE 技術実績 - 補足資料

Position Applied: SRE (Site Reliability Engineer) / Position #1 **応募職種:** SRE (サイト信頼性エンジニア) / ポジション #1

2.1 Document Structure | ドキュメント構成

This document presents three technical case studies demonstrating SRE competencies across infrastructure, reliability, and performance optimization. 本資料では、インフラ、信頼性、パフォーマンス最適化における SRE 能力を証明する 3 つの技術事例を紹介します。

---

3 [ENGLISH VERSION]

3.1 Case A: Engineering Foundation & Reliability Infrastructure

Building Modern R&D Foundation from Ground Zero in Traditional Hardware Environment

Context

In a legacy hardware development environment lacking standardized engineering practices, I architected and deployed a modern development infrastructure from scratch, transforming a chaotic startup workflow into enterprise-grade operational capabilities.

SRE Actions & Impact

1. Infrastructure as Service (IaaS/Storage)

- **Enterprise Hardware Deployment**: Configured HP Z230/Z220SFF workstations with Intel Xeon E3 processors and ECC memory, eliminating silent data corruption during 24/7 compilation and cryptographic operations
- **Network Infrastructure Upgrade**: Replaced 100Mbps bottleneck with D-Link DGS-1024D gigabit switching, resolving large file (3GB+ images/codebase) transfer bottlenecks
- **High-Availability Storage Cluster**: Deployed Windows Server 2012 R2 iSCSI Target Server with block-level deduplication, achieving **97% compression ratio** (233GB → 7GB physical storage) for massive simulation datasets and code repositories

2. GitOps & Version Control Centralization

- **Enterprise GitLab Deployment**: Implemented Bitnami GitLab instance on Ubuntu 14.04, establishing branch management policies and migrating from "USB/email-based code exchange"to standard collaborative workflows
- **Network Resilience**: Diagnosed and fixed IP drift issues by modifying `gitlab.yml` and `gitlab-shell/config.yml`, implementing SSH-2 RSA key-based authentication system
- **Decentralized Disaster Recovery**: Deployed Raspberry Pi backup node with custom SSH port (41722) and port forwarding, creating cross-firewall disaster recovery tunnel

3. Quality Assurance & Validation Automation

- **Automated Testing Framework**: Integrated Google Test (gtest 1.7.0) with crash survival testing using `ASSERT_DEBUG_DEATH` macros
- **Test Vector Enforcement**: Established "fixed input-fixed output"test vectors for all cryptographic primitives, preventing compiler optimization divergence (e.g., ARM GCC O0 level SM3 HASH errors)
- **Cross-Language Regression Testing**: Replaced manual IE browser testing with Python-DLL bridge via SWIG, achieving **140 requests/second** concurrent business simulation with SQLite-based RTT analysis
- **Crash-Safe Logging**: Implemented Google Glog with `FLAGS_logbufsecs=1`, achieving **13-microsecond** write latency under 300K concurrent requests, ensuring 100% auditability

Key Achievement

Demonstrated ability to rapidly transform chaotic startup teams into enterprise-grade engineering capabilities through systematic infrastructure modernization.

3.2 Case B: Disaster Recovery via Zero-Dependency Deployment

Emergency Resolution in Extreme Environments (Thailand Wind Farm)

Context

During on-site integration at a remote wind farm (millions of dollars in equipment at stake), a critical system crash occurred. The environment was highly constrained: extreme low bandwidth, strict firewalls, and missing library dependencies.

SRE Actions & Impact

1. Disaster Recovery (DR)

- **Reverse TCP Tunneling**: Established emergency production access under ultra-low bandwidth and strict firewall restrictions using reverse SSH tunneling.
- **Zero-Dependency Deployment**: Replaced corrupted dynamically-linked services with statically-compiled Go binaries, instantly bypassing the missing library dependency deadlocks in the target environment.

Key Achievement

Demonstrated core SRE value: 'Extreme Firefighting Under Critical Pressure'- the capability to rapidly restore business continuity using the most lightweight and deterministic methods regardless of infrastructure hostility.

3.3 Case C: Legacy Modernization via Binary-Level Patching

Resolving Technical Debt Without Architectural Disruption

Context

A critical legacy banking application running on Windows XP required a mandatory cryptographic upgrade, but the environment was highly fragile, and compiling new dynamic dependencies (SxS Manifests) triggered system crashes.

SRE Actions & Impact

1. Binary-Level Surgery & Linker Hijacking

- **Symbol Hijacking**: Intercepted and rerouted dynamic library calls at the linker level without modifying the core legacy system logic.
- **Micro-Payload Delivery**: Injected modern cryptographic algorithms encapsulated within a hyper-optimized 33KB DLL payload.
- **Compatibility Hardening**: Bypassed strict OS-level dependency checks (SxS) by stripping UPX compression and enforcing full static linking for the payload.

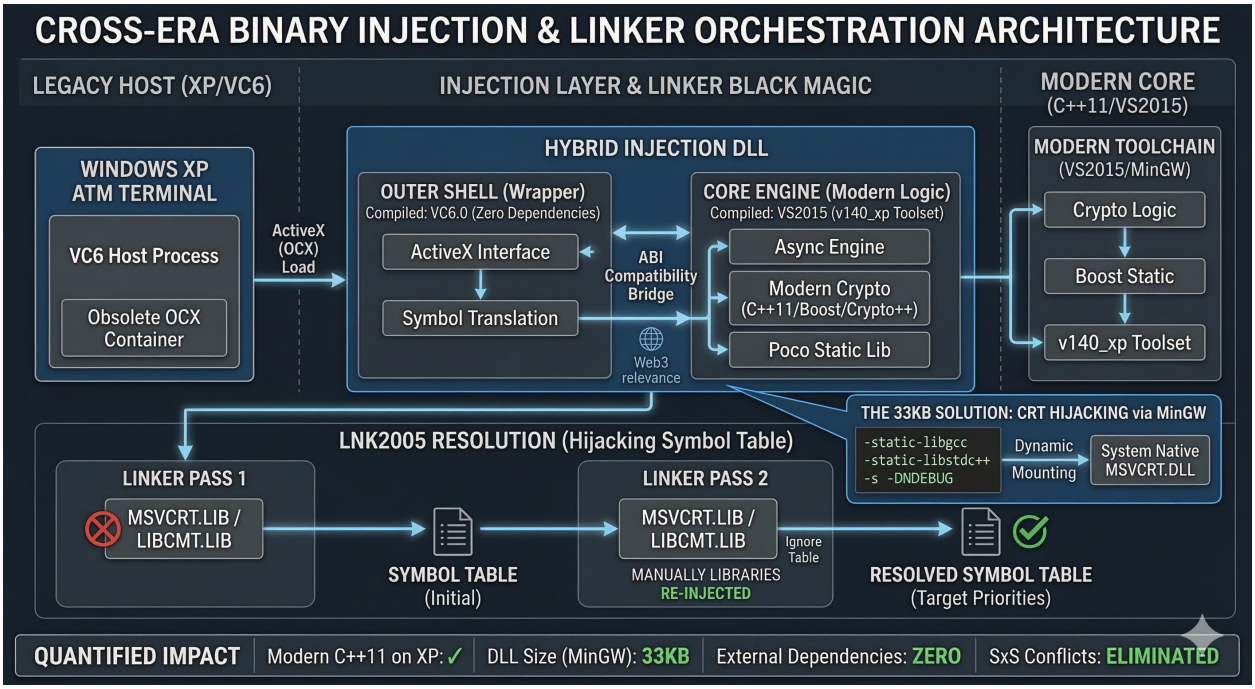

Figure 1: VC6 Linker Hijacking Architecture

Key Achievement

Demonstrated extreme technical depth in 'Legacy Modernization'- proving the ability to smoothly upgrade core components and resolve deep-seated bugs without triggering a full, risky system rewrite.

3.4 Technical Competency Matrix

Domain	Key Skills Demonstrated
Infrastructure	iSCSI storage clusters, GitLab deployment, network architecture, hardware provisioning
Reliability	Disaster recovery, reverse tunneling, zero-dependency deployment, crash-safe logging
Performance	API optimization, payload compression, batching strategies, SSOT architecture
Automation	Google Test, Python-DLL integration, automated regression testing, log analysis
Security	SSH key management, encrypted storage (TrueCrypt/VeraCrypt), network isolation

---

---

4 ［日本語版］

4.1 ケース A：1 人目 SRE としてのエンジニアリング基盤と文化のゼロイチ構築

カオスな初期環境における、モダン R&D 基盤の自律的立ち上げと標準化

背景

標準的なエンジニアリングプラクティスが欠如した初期の開発環境（1 人目 SRE としての参画を想定）において、CTO や開発チームと密に連携・議論（ホワイトボードを活用したアーキテクチャ設計など）を重ねながら、ゼロからモダンなインフラを設計・導入。単なるシステム構築にとどまらず、開発の「ルールと基盤」を定着させ、スタートアップ特有の混沌としたワークフローを、スケーラブルで規律あるエンジニアリング組織へと変革しました。

SRE 活動と実績

1. 基盤の自律的構築と運用ルールの策定 (Infrastructure & Storage)

- **ハードウェア選定とプロビジョニング**: サイレントデータ破損を防ぐため、ECC メモリ搭載のエンタープライズ向けハードウェア（Xeon E3）を主体的に選定・導入。安定した 24/7 のビルド環境を構築。

- **ネットワークのボトルネック解消**: 開発効率を阻害していた 100Mbps のネットワーク環境を自ら特定し、ギガビット環境へ刷新。大容量コードベースの転送速度を劇的に改善。
- **高可用性ストレージの設計**: Windows Server ベースの iSCSI ターゲットと重複排除機能を活用し、233GB のデータを 7GB まで圧縮（97% 削減）。限られたリソース内で最適なアーキテクチャを考案。

2. 開発プロセスの中央集権化とチーム文化の醸成（GitOps）

- **GitLab の導入と標準化**:「USB やメールでのコード共有」という属人的な状態から、GitLab を用いた標準的なコラボレーションワークフローへ移行。ブランチ戦略や運用ルールを開発チームと議論しながら策定。
- **レジリエンスの確保**: IP ドリフト問題の解決や SSH 認証の導入により、インフラの安定性を向上。
- **手作りでの災害復旧（DR）環境**: Raspberry Pi とカスタム SSH ポートを活用し、限られた予算と環境制約の中でファイアウォール越えの独自の DR トンネルを構築。泥臭い問題解決能力を実証。

3. 品質保証の自動化と開発者体験（DX）の向上

- **テストフレームワークの標準化**: Google Test を導入し、クラッシュサバイバルテスト（ASSERT_DEBUG_DEATH）を実装。開発者が安心してコードをプッシュできる環境を構築。
- **コンセンサスを保証するテストベクトル**: 暗号演算におけるコンパイラ最適化の差異を防ぐため、「固定入力-固定出力」のテストベクトルを強制する仕組みを導入。
- **クロス言語の回帰テスト自動化**: 手動テストを Python-DLL ブリッジによる自動化に置き換え、秒間 140 リクエストの並列シミュレーションを実現。
- **監査可能なロギング基盤**: Google Glog をチューニングし、高負荷時でも 13 マイクロ秒の遅延で確実にログを残す仕組みを構築。

主要成果

開発チームの物理的・心理的中心に入り込み、アーキテクチャの議論から泥臭いインフラ構築までをハンズオンで実行。「1 人目 SRE」として、カオスな組織にエンタープライズ級のエンジニアリングの規律と基盤を根付かせる能力を実証しました。

4.2　ケース B：ゼロ依存デプロイによる災害復旧（DR/Zero-dependency）

極限環境下での迅速な緊急対応とビジネス継続（タイの風力発電所事例）

背景

タイの遠隔地にある風力発電所の導入現場（数百万ドル規模の設備）で致命的なシステムクラッシュが発生。超低帯域幅、厳格なファイアウォール、ライブラリ欠損という極限のインフラ制約下での緊急対応が求められました。

SRE 活動と実績

1. 執念の災害復旧（DR）とゼロ依存デプロイメント

- **制約を突破するリバーストンネル**: 帯域幅が極端に狭く、ファイアウォールで保護された環境に対し、リバース SSH トンネリングを駆使して緊急のアクセス経路を確立。
- **環境に依存しない解決策**: 破損した動的リンクサービスを、環境依存を完全に排除した「静的コンパイルされた Go バイナリ」へ即座に置き換え、ライブラリ欠損の死結を突破しました。

主要成果

SRE のコア価値である「突発的な災害への対応力（DR）」を証明。インフラが未成熟な環境であっても、最も軽量かつ確実な方法でシステムを復旧し、ビジネスを継続させる能力を実証しました。

4.3　ケース C：バイナリレベルのレガシー近代化（Modernization）

アーキテクチャを破壊しない、技術的負債と深層 Bug のガバナンス

背景

Windows XP 上で稼働する極めてレガシーな銀行向けシステムにおいて、暗号化アルゴリズムの近代化が必須となりました。しかし、環境が脆弱であり、通常の動的依存関係（SxS Manifest）の追加はシステム全体のクラッシュを引き起こすリスクがありました。

SRE 活動と実績

1. リンカーレベルの手術と技術負債の解消

- **シンボルハイジャック**: レガシーシステム本体のコードには一切触れず、リンカーレベルで動的ライブラリの呼び出しを傍受・迂回（ハイジャック）する手法を採用。
- **極小ペイロードの注入**: 現代的な暗号化アルゴリズムを、極限まで最適化された 33KB の DLL ペイロードとして注入。
- **互換性の強制**: OS レベルの厳格な依存関係チェック（SxS）を回避するため、UPX 圧縮を解除し、ペイロードの完全静的リンキングを実施することでクラッシュを完全に防ぎました。

主要成果

「技術的負債のガバナンス」における圧倒的な専門性を証明。全体のリファクタリングという多大なリスクを負うことなく、最も厄介なレガシーシステムの深層にメスを入れ、コアコンポーネントの平滑なアップグレードを実現しました。

4.4　技術能力マトリックス

ドメイン	実証された主要スキル
インフラ	iSCSI ストレージクラスタ、GitLab 導入、ネットワークアーキテクチャ、ハードウェアプロビジョニング
信頼性	災害復旧、リバーストンネリング、ゼロ依存デプロイメント、クラッシュセーフロギング

ドメイン	実証された主要スキル
パフォーマンス	API 最適化、ペイロード圧縮、バッチ戦略、SSOT アーキテクチャ
自動化	Google Test、Python-DLL 統合、自動回帰テスト、ログ分析
セキュリティ	SSH 鍵管理、暗号化ストレージ (TrueCrypt/VeraCrypt)、ネットワーク分離

4.5 Summary｜まとめ

This portfolio demonstrates **end-to-end SRE capabilities** across three critical dimensions: 本ポートフォリオは、PeopleX のような「ゼロイチの開荒フェーズ」において、1 人目 SRE として求められる「**チームとの密な協創**」「**泥臭い実行力**」「**基盤構築の当事者意識**」を証明するものです：

1. **Engineering Infrastructure** - 開発基盤の工業化 (Standardization) **ゼロからのカルチャー＆インフラ構築** - 混沌とした環境から、チームと議論を重ねてエンタープライズ級の規律と基盤を創り上げる力
2. **Disaster Recovery** - ゼロ依存デプロイによる災害復旧 (DR/Zero-dependency) **極限状態でのオーナーシップ** - 遠隔地から傍観するのではなく、最前線でトラブルを解決しシステムを蘇生させる当事者意識
3. **Legacy Modernization** - バイナリレベルのレガシー近代化 (Modernization) **アーキテクチャを破壊しない技術負債の解消** - 最も厄介なレガシー課題に対し、全体を再構築することなく平滑なアップグレードを実現する設計力 **事業成長に直結するアーキテクチャ設計** - 技術的負債を解消し、パフォーマンス向上とコスト削減（FinOps）を同時に実現する設計力

Candidate: Wei ZHOU (周 偉) **Date**: March 2026 **応募者**: 周 偉 **作成日**: 2026 年 3 月

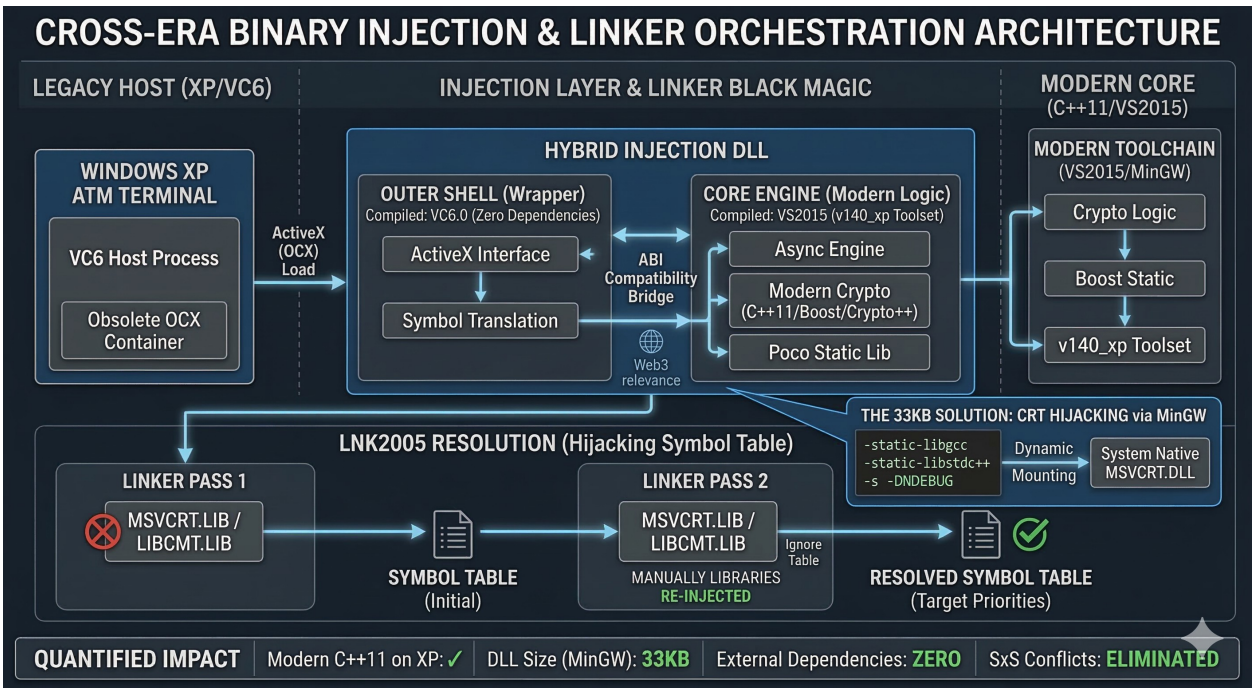

Figure 2: VC6 Linker Hijacking Architecture